

Bitcoin Mining



PRESENTED BY:

Hasitha Peter Inapuri
Jin Hao Dylan Loh
Pyae Sone Oo (Morris)
Qiyue Wang (Martin)

PROJECT OVERVIEW

SHA-256 ALGORITHM:

PROJECT AIM:

“IMPLEMENTING A BASIC HARDWARE ACCELERATED BITCOIN MINER”

```
```python def mine_bitcoin(header):  
 # Loop until we found a good nonce or we run out of time or killed.
 while(timeout == False):
 # Increment the nonce (garbage field we are allowed to set to
 # influence the output hash)
 header.nonce += 1
 # Return our golden nonce if we find it!
 if (sha256d(header) < header.target): break return header

def sha256d(header): sha256(sha256(header)) ```
```

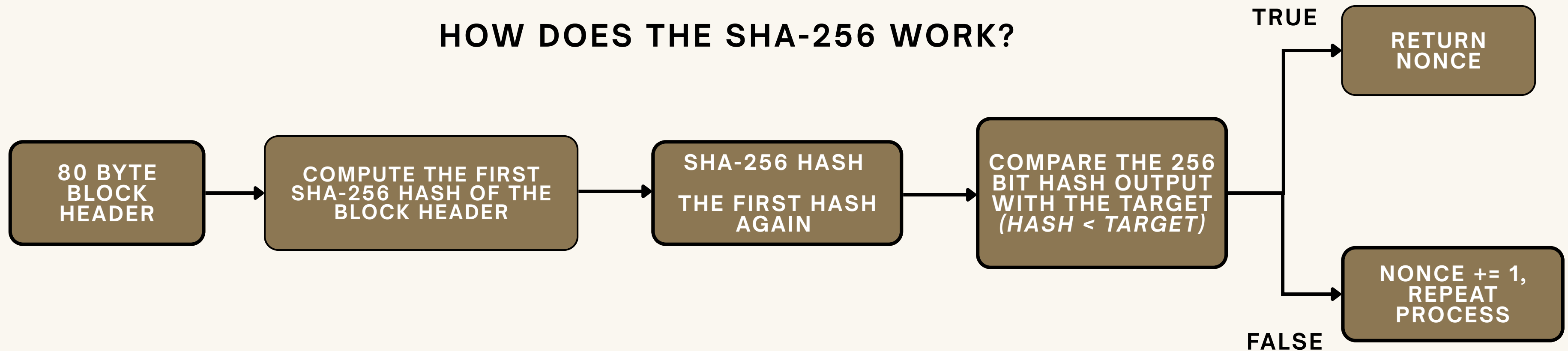
# OBJECTIVES

- BUILD A SHA-256D MINING CORE IN VITIS HLS AND RUN IT ON THE KV260 BOARD
- KEEP THE WHOLE NONCE SEARCH INSIDE THE PROGRAMMABLE LOGIC. THE ARM CORES ONLY HAND OUT WORK AND VERIFY ANSWERS.
- MEASURE THE HASH RATE AND EFFICIENCY (ENERGY PER HASH) AGAINST A TUNED SOFTWARE MINER ON A MODERN CPU AND ON THE KV260'S ARM CORES.
- WORKING TARGET: AT LEAST 20X THE SOFTWARE HASH RATE (TO BE FIRMED UP AFTER PROFILING)



# ALGORITHM OVERVIEW

## HOW DOES THE SHA-256 WORK?



# ALGORITHM OVERVIEW

## THE MINING ALGORITHM

- Midstate reuse:
  - Constant header data is precomputed on the host to avoid padding the algorithm latency
- The FPGA tests candidate nonces in chunks of up to 2048
- Each candidate nonce undergoes SHA-256D

### INPUT:

80-byte Bitcoin block header  
256-bit target  
nonce\_start  
nonce\_count

### HOST PREPROCESSING:

Compute SHA-256 midstate from header bytes 0-63  
Extract fixed header bytes 64-75 as tail words  
Convert target into 32-bit words

### FPGA MINER:

for each chunk of up to 2048 nonces:

for each nonce in chunk:  
nonce = nonce\_start + current\_offset

Complete first SHA-256 using:

saved midstate  
fixed tail words  
current nonce

Perform second SHA-256

if final\_hash < target:  
mark nonce as valid

Scan chunk results for the lowest valid nonce

if valid nonce found:  
return winning nonce

return "not found"

## PROJECT OVERVIEW

# ALGORITHM OVERVIEW

## THE SHA-256 HASH LOOP

- Processes data in 512-bit blocks
- Performs 64 hashing rounds per block
- Uses message expansion and bitwise operations to update the hash state
- Produces a 256-bit hash

```
SHA256(message):
```

```
 Pad message to a multiple of 512 bits
```

```
 Initialise hash state
```

```
 FOR each 512-bit block:
```

```
 Split block into 16 × 32-bit words
```

```
 FOR round = 0 TO 63:
```

```
 IF round >= 16:
```

```
 Generate next message-schedule word
```

```
 Calculate round values using:
```

- rotations and shifts
- Choose function
- Majority function
- round constant

```
 Update hash state
```

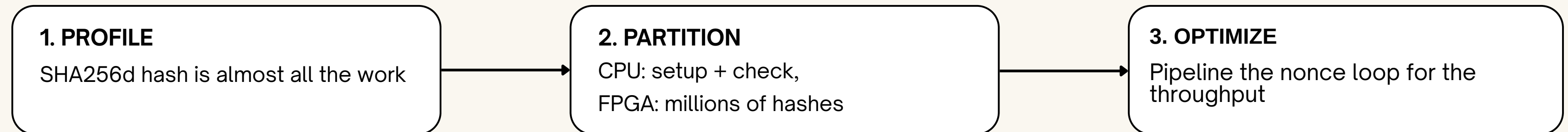
```
 Combine final hash state
```

```
 RETURN 256-bit hash
```

## DESIGN

# Solution Approach

## GAME PLAN



## KEY TECHNIQUES

### Midstate

The first 64 header bytes never change with the nonce. Hash the block **once on the CPU**, so the kernel only redoes the small changing part.

Half the work per nonce

### Fully unroll the hash

Unroll every SHA256 round, so the whole double hash becomes **one long hardware pipeline** instead of a loop.

A deep pipeline

### One nonce per cycle

The deep pipeline takes **a new nonce every clock cycle**. The initiation interval value is 1 which means this is the fastest a pipeline can run.

II = 1

**TEAM'S USE OF AI:** We used AI throughout the project, mostly Claude Code for software baseline. The optimization ideas come from the team and Tony's remarks.

## DESIGN

# HW/SW Architecture

AMD Kria KV260 SOC

### SW ARM PS (host)

1. Build 80-byte block header
2. Decode **nBits** → 256-bit target
3. Compute the **midstate** (hash of block 1)
4. Start the kernel and wait
5. Reade back the nonce and the found flag
6. Verify the winner

Inputs[76 bytes]  
*midstate[32]*  
*tail[12]*  
*target[32]*

Results[8 bytes]  
*found\_nonce[4]*  
*found[4]*

### HW FPGA PL (kernel)

Fully unrolled (II=1)

For each nonce:

- assemble block 2 ( tail + nonce )
- SHA-256 compress x 2 (**sha256d**)
- compare **digest** < **target** , set a match flag

*After the loop: scan the flags, output the winning nonce*

AXI BUS

AXI-Lite : control and start/done

AXI-master : data buffers to/from DRAM

RESULTS

# Performance Evaluation

## *HLS synthesis*

Performance & Resource Estimates											
MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	
mine (1)	5.069E7		10,137,744		no	function	30	500	52,378	101,091	
CHUNK_LOOP (2)	5.069E7	2,475	-	4,096	no	loop					
mine_Pipeline_HASH_CHUNK (1)	1.106E4		2,049		loop pipeline stps	function	0	500	47,695	95,148	
HASH_CHUNK (2)	1.105E4	163	1	2,048	yes	loop					
mine_Pipeline_SCAN_CHUNK_PARALLEL (1)	1.295E3		257		loop pipeline stps	function	0	0	77	419	
SCAN_CHUNK_PARALLEL	1.285E3	3	1	256	yes	loop					

- 200MHz clock frequency
- HASH\_CHUNK achieved II = 1
- New nonce can be achieved every clock cycle
- Resource utilization:
  - BRAM = 30~(10%)
  - DSP = 500~(40%)
  - FF = 52378~(22%)
  - LUT = 101091~(86%)

## *KV260 Board results*

```
=====
PL HARDWARE (optimized kernel @ 200 MHz)
 nonces/run : 268435456
 runs : 5
 median kernel: 2.047930 s
 median total : 2.047937 s
 hash rate : 131.076 MH/s

ARM PS SOFTWARE (4-threaded SHA256d)
 nonces : 2000000
 time : 2.360653 s
 hash rate : 0.847 MH/s
 checksum : 0xf43022b934b0c

SPEEDUP hardware / software : 154.71x
=====
TEST PASSED
```

- **268435456** nonces processed per run
- Median Kernel time = **2.048s**
- FPGA hash rate: **131.076MH/s**
- 4-thread ARM hash rate: **0.847 MH/s**
- FPGA achieved **154.71× speedup**

RESULTS

# Discussion

- The board implementation achieved a throughput of 131.076 MH/s, providing a 154.71x speed up over our ARM PS software
- **Measured vs theoretical:** the official board implementation reach approximately 65.5% of the theoretical throughput.
- Best performance results required significantly high resource utilisation, particularly 86% LUT utilisation, making LUT the primary resource constraint.
- The power usage increased from 3.63W at an ideal state to 4.47W when the board was running, an increase of only 0.84W(~23%).
- $II = 1$  allows a nonce to enter the SHA-256d pipeline every clock cycle, giving a theoretical throughput of 200 MH/s.

```
===== 2026-08-09 05:33:45 =====
Power Utilization
SOM total power : 3630 mW
SOM total current : 718 mA
SOM total voltage : 5042 mV

AMS CTRL
System PLLs voltage measurement, VCC_PSLL : 1202 mV
PL internal voltage measurement, VCC_PSBATT : 716 mV
Voltage measurement for six DDR I/O PLLs, VCC_PSDDR_PLL : 1793 mV
VCC_PSINTFP_DDR voltage measurement : 840 mV

PS Sysmon
LPD temperature measurement : 32 C
FPD temperature measurement (REMOTE) : 32 C
VCC PS FPD voltage measurement (supply 2) : 845 mV
PS IO Bank 500 voltage measurement (supply 6) : 1781 mV
VCC PS GTR voltage : 858 mV
VTT PS GTR voltage : 1799 mV

PL Sysmon
PL temperature : 30 C
```



```
===== 2026-08-09 05:34:43 =====
Power Utilization
SOM total power : 4470 mW
SOM total current : 871 mA
SOM total voltage : 5041 mV

AMS CTRL
System PLLs voltage measurement, VCC_PSLL : 1202 mV
PL internal voltage measurement, VCC_PSBATT : 718 mV
Voltage measurement for six DDR I/O PLLs, VCC_PSDDR_PLL : 1791 mV
VCC_PSINTFP_DDR voltage measurement : 841 mV

PS Sysmon
LPD temperature measurement : 32 C
FPD temperature measurement (REMOTE) : 32 C
VCC PS FPD voltage measurement (supply 2) : 840 mV
PS IO Bank 500 voltage measurement (supply 6) : 1780 mV
VCC PS GTR voltage : 857 mV
VTT PS GTR voltage : 1796 mV

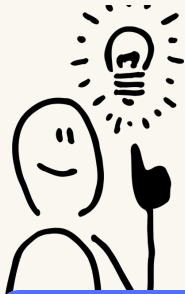
PL Sysmon
PL temperature : 32 C
```

• Power when idle: 3.63W

• Power when running: 4.47W

## CONCLUSION

# Challenges & Limitations



### WHAT WORKED

- Host-side midstate preprocessing removed repeated SHA-256 work for every nonce.
- $II = 1$  pipelining + rolling 16-word schedule achieved high throughput without the excessive resource usage of full unrolling.
- 2048-nonce chunks + 8-way result scan reduced pipeline and search overhead.
- Hybrid DSP/LUT mapping + post-route physical optimisation allowed the final design to meet the required 200 MHz.
- Hardware correctness verified on KV260.



### WHAT DIDN'T

- Initial  $II \approx 231$  design: too sequential, severely limiting throughput.
- Brute-force  $II = 1$  implementation: required ~290% LUTs, so it could not fit on the K26.
- No DSP bindings: LUT usage increased to ~97%, leaving insufficient routing headroom.
- DSP-heavy implementation: fit the FPGA but failed 200 MHz timing due to long DSP-related critical paths.
- Short hardware benchmarks: XRT/runtime overhead dominated execution time, giving misleadingly low MH/s results.

## CONCLUSION

# With given more time...

- **Profile earlier before aggressive optimization**
  - Identify the true bottlenecks before spending time on resource-heavy  $l=1$  approaches.
- **Explore more SHA-specific preprocessing:**
  - Pre-compute additional nonce-independent message-schedule values to further reduce FPGA work.
- **Investigate multiple parallel hashing pipelines:**
  - use remaining FPGA resources to process multiple nonces per cycle while maintaining 200 MHz.
- **Improve timing closure earlier:**
  - consider DSP/fabric placement and routing effects during HLS optimization, rather than only after synthesis.
- **Build a more rigorous benchmark framework:**
  - separate XRT launch overhead from sustained kernel throughput and compare against an optimized ARM baseline.
- **Automate design-space exploration:**
  - systematically test chunk size, scan width, DSP mapping and pipeline configurations instead of tuning them manually.

# Thank You



**PRESENTED BY:**

Hasitha Peter Inapuri  
Jin Hao Dylan Loh  
Pyae Sone Oo (Morris)  
Qiyue Wang (Martin)